TensorGuard: Static Refinement Verification for PyTorch Tensor Programs

TensorGuard Authors

June 3, 2026

Abstract

TensorGuard is a static verifier for PyTorch models that turns tensor-program well-formedness into refinement constraints. It checks shape, device, dtype, train/eval phase, stride, permutation, and gradient-flow facts before training or export begins, while preserving an explicit three-valued soundness contract: when the verifier cannot prove a claim inside its supported fragment it reports UNKNOWN rather than silently passing. The implementation combines a source frontend, a `torch.fx`/export-oriented graph frontend, a reduced product of tensor abstract domains, Z3-backed safety checks, CEGAR-style predicate discovery, differential conformance tests against live PyTorch, Lean proof artifacts for core rules, and reproducible empirical harnesses.

The repository is not merely a prototype pass over toy models. It contains operator transfer functions for everyday PyTorch programs, frontends for real `nn.Module` code, integration hooks for CLIs, GitHub Actions, SARIF, watch mode, Jupyter, decorators, framework adapters, deployment gates, ONNX shape-inference round trips, model-checkpoint schema gates, optimizer-state resume gates, CUDA graph capture diagnostics, and artifact scripts that regenerate precision/recall, latency, ablation, coverage, conformance, fuzzing, mutation, and benchmark evidence. This paper describes the system as shipped in the repository, with emphasis on the engineering and evaluation choices that make TensorGuard a credible research artifact and a useful developer tool.

1 Problem and Claim

Most deep-learning correctness tools see tensor bugs too late. A model can import successfully, type-check as Python, instantiate its modules, and even reach a training loop before a wrong channel count, a bad attention projection, an accidental CPU buffer, a mismatched dtype, or a train/eval phase dependency raises a runtime exception. Worse, some failures are silent: a detached tensor can sever gradient flow, a broadcast can mask an intended batch axis, or an export pipeline can accept a graph whose serving contract no longer matches the model author's intent.

TensorGuard attacks these failures as static contract violations. A tensor value is modeled with refinement information such as

$$\{v : \text{Tensor} \mid \mathsf{Shape}(v) = (B, C, H, W) \wedge \mathsf{Device}(v) = \text{cpu} \wedge \mathsf{DType}(v) = \text{float32}\}.$$

The verifier propagates these facts through constructors, module calls, functional operators, reshapes, broadcasts, indexing, attention blocks, recurrent modules, normalization layers, torchvision v2 image transforms, sparse layouts and sparse-dense kernels, distributed wrappers, and deployment gates. At each operation it asks whether the preconditions of the transfer function are entailed by the current abstract state. Concrete contradictions become high-confidence bugs. Unknown or out-of-fragment regions become abstentions with reasons, not fake successes.

The central claim of the repository is practical: a tensor verifier can be soundly conservative and still catch real bugs early across modern PyTorch workflows. The codebase supports that claim with four pillars.

1. A precise public contract that separates SAFE, UNSAFE, and UNKNOWN outcomes, with a strict "sound" mode for CI.
2. A broad operator and frontend implementation that handles real modules, not only a calculus in a paper.
3. A reproducible evaluation suite with real-bug corpora, differential testing, mutation tests, benchmark scripts, ablations, dashboards, and baseline comparisons.
4. A formalization and conformance story: Lean artifacts for core transfer rules, Z3 encodings for symbolic constraints, and live PyTorch oracles for operator behavior.

This is a tool paper: the interesting contribution is the system boundary, not a single transfer function. The repository demonstrates how a static analysis tool can be packaged so that reviewers, users, and future contributors can inspect the guarantee, reproduce the headline evidence, and extend the coverage without weakening the trust story.

2 Architecture

TensorGuard is organized as a pipeline from Python source or captured graphs to a multi-domain safety decision. The source path parses `nn.Module` classes, extracts constructor parameters and layer definitions, walks `forward`, and emits a computation graph. The graph path consumes `torch.fx`, `torch.export`, and integration-specific traces where available. Both paths feed a common verifier built around symbolic tensor states.

Each state maps tensor names to abstract facts. Shape facts are tuples of concrete integers or symbolic names; device facts range over CPU and CUDA devices; phase facts track train/eval context; dtype

facts model promotion and casts; gradient facts track whether a value can carry gradient to downstream parameters. Several later steps add stride and permutation information so viewability and layout-sensitive operators can be checked without collapsing to opaque shapes. These domains form a reduced product: a violation in one domain can be explained with facts from the others, while disabled domains are gated out of the solver rather than filtered only at the end.

The pipeline has four visible outputs. First, it returns structured bugs with locations, severities, confidence values, and optional counterexamples. Second, it returns a verdict: SAFE, UNSAFE, or UNKNOWN. Third, it can emit developer-facing diagnostics, including source snippets, inferred-vs-expected shapes, explanations, and autofix suggestions for mechanical cases. Fourth, it can emit machine-readable reporters: JSON, JUnit XML, SARIF 2.1.0, GitHub annotations, coverage artifacts, dashboards, and certificates.

2.1 Frontends

The source frontend is deliberately useful on unannotated Python. It recognizes `nn.Linear`, convolution families, pooling, normalization, attention, activation layers, containers, helper methods, subclassing patterns, and functional calls such as `F.linear`, `F.pad`, `F.interpolate`, and `F.scaled_dot_product_attention`. It records scalar constructor attributes such as head counts and hidden sizes, so common model code like `x.view(B, T, self.num_heads, D)` can be checked against the same constants used by the layers.

The graph frontends target execution-oriented workflows. `torch.fx` support traces real modules and preserves enough source mapping for diagnostics. `torch.export` and Dynamo-oriented paths let TensorGuard run as a pre-pass for export and compile pipelines. The repository also includes integration points for guarded `torch.compile`, AOTInductor packaging, ONNX export, GGUF/llama.cpp-oriented checks, model checkpoint load preflights, optimizer-state load/resume preflights, CUDA graph capture eligibility diagnostics, source-mapped Dynamo/export graph-break attribution, framework hooks, and deployment request/response schemas. The ONNX path validates structural checker and opset contracts, then round-trips output shapes predicted from captured `torch.export` metadata against `onnx.shape_inference` wherever both sides infer concrete dimensions. These paths are important because many production tensor bugs appear only when a model is compiled, exported, quantized, sharded, or served.

2.2 Verifier

The verifier performs stepwise symbolic propagation. For each computation step it updates the tensor state, emits safety conditions, and asks Z3 whether a violation is forced. Concrete mismatches, such as a `Linear` layer expecting 512 features but receiving 256, are immediate. Symbolic mismatches are handled conservatively: if the verifier can prove incompatibility under the known assumptions it reports a bug; if the result depends on an unconstrained symbol, it abstains or keeps the dimension symbolic.

The same engine also supports CEGAR. When the current abstraction is too weak, TensorGuard proposes additional shape predicates, reruns the proof attempt, and records the discovered predicates as metadata. If the predicates jointly imply an impossible input contract, the conflict is promoted to a real `cegar_refined_contract` bug rather than hidden as an internal note. The repository includes convergence measurements and Lean-checked statements for the predicate-bound argument.

2.3 Soundness Modes

The user-facing contract is intentionally explicit. In `sound` mode, TensorGuard reports SAFE only when the program remains inside the verifiable fragment and all relevant transfer functions are at an acceptable

confidence level. Unsupported layers, opaque data-dependent control flow, or heuristic operators make the verdict UNKNOWN unless a concrete bug is found. In `balanced` mode the tool remains useful for day-to-day development by allowing more best-effort reasoning, while still surfacing abstentions. In `heuristic` mode users can opt into permissive exploratory checks. The mode changes the verdict boundary; it does not hide discovered bugs.

3 Tensor Semantics and Transfer Functions

A transfer function is the contract for one operator family. It consumes input abstract facts and produces output facts or a violation. TensorGuard tags each operator as complete, sound, or heuristic. Complete rules are exact shape-preserving families such as pointwise activations. Sound rules enforce well-defined structural contracts, even if the implementation abstains on hard symbolic cases. Heuristic rules are used when output rank or size depends on runtime values, or when the rule intentionally approximates a large behavior surface.

The repository includes transfer functions and tests for a wide range of PyTorch constructs. The core structural set covers matrix multiplication, batched matrix multiplication, linear layers, convolution and transpose convolution families, pooling, flatten, reshape/view with product reasoning, broadcasting, reductions, cat/stack-family aliases, PyTorch-faithful padding, interpolation/upsampling, transpose/permute, squeeze/unsqueeze, `movedim`/`swapaxes`, `roll`/`rot90`/`flip`, `repeat_interleave`/`tile`, tensor broadcasting helpers, fold/unfold image-column transforms, embedding, dtype casts, device moves, and stochastic tensor factories whose shapes are seed-independent. Extended coverage includes einsum parsing, advanced indexing, gather/scatter, index select, masked select, `take_along_dim`, `argsort`, `sort`/`topk`/ `kthvalue`, arg reductions, exact `nn/F.embedding_bag` contracts for offsets, per-sample weights, pooling modes, and TorchRec-style jagged pooled embeddings, sparse COO/CSR/CSC/BSR/BSC layout invariants, sparse-dense `mm`/`addmm`, CSR `sampled_addmm`, sparse softmax/coalesce, layout conversions, FFT/complex view contracts, exact `torch.linalg` contracts for `solve`, inverse, Cholesky, SVD, eigendecomposition, and QR, named tensor alignment, vmap/functorch batch dimensions, `torch.func` autodiff wrappers (`grad`, `jacrev`, `jacfwd`, `jvp`, and `vjp`), distribution batch/event/log-prob shapes, grid sampling, affine grids, source-level `torchvision.transforms.v2` tensor transforms (`Resize`, crops, `Pad`, `Normalize`, flips, and PIL-only abstention), multi-head attention, scaled dot-product attention, training loss contracts for `CrossEntropyLoss`, `NLLLoss`, `MSELoss`, `BCEWithLogitsLoss`, and `KLDivLoss`, exact RNN/LSTM/GRU output and returned-state contracts, LoRA adapter compatibility, training-loop checks, quantization/export safety, mixed-precision/autocast dtype-divergence gates, FSDP/FSDP2/DTensor sharding, explicit pipeline-stage boundary contracts, tensor-parallel sharding checks, model `state_dict` schema compatibility, optimizer-state checkpoint compatibility, CUDA graph capture eligibility, and TorchServe/FastAPI serving schemas.

Recent stack-family work follows the same standard. `torch.stack`, `hstack`, `vstack`, `dstack`, `column_stack`, and `row_stack` are modeled with PyTorch's singleton and rank-promotion corner cases, then checked by tests that run the real operators and compare both output shapes and static error parity.

Axis-structural operators are now treated with the same precision rather than hidden behind a generic shape-preserving fallback. `squeeze` and `unsqueeze` use PyTorch's exact rank rules, including tuple-dim `squeeze` and the legal $[-\text{rank}-1, \text{rank}]$ `unsqueeze` interval. `movedim`/`moveaxis` use the ATen axis-order algorithm, so multi-axis placements preserve symbolic dimension identity instead of merely preserving rank. `swapaxes`/`swapdims` share the transpose transfer. `roll`, `rot90`, and `flip` preserve or permute shapes while still refuting statically invalid axis contracts such as duplicate flip axes, mismatched roll arity, or invalid rotation axes. The regression suite compares source and FX extraction against live PyTorch, exhaustively checks rank-4 `movedim` placements, and proves that symbolic dimensions do not create false structural bugs.

Repeat and broadcast helpers are modeled as first-class structural transfers, not as generic repeats or activations. `repeat` and `tile` use their distinct eager-PyTorch rank rules: `repeat` requires enough factors for the input rank, while `tile` left-pads factors before multiplying axes. `repeat_interleave` preserves the flattening behavior of `dim=None`, statically checks `output_size` when the repeated extent is provable, and otherwise keeps a fresh symbolic extent instead of claiming a false proof for data-dependent repeat vectors. `torch.broadcast_tensors` now gives every returned tensor the exact joint broadcast shape,

while source-level `torch.broadcast_shapes` is carried as a shape value that can feed `expand`. Focused source and FX tests compare these contracts against live PyTorch successes and exceptions.

Sparse tensors are checked beyond construction. TensorGuard verifies sparse-dense `torch.sparse.mm` and `addmm` for COO/CSR/CSC matrices, exact one-way `addmm` input broadcasting, CSR `sampled_addmm` including batched dense operands, COO-only sparse softmax and coalescing, `to_dense`, and `to_sparse_{coo,csr,csc,b` conversions with blocked-layout divisibility. The parity tests compare every successful output to live PyTorch via dense shape equality and keep unsupported BSR/BSC sparse matrix kernels as explicit non-passes rather than build-dependent silent approvals.

3.1 Exact Index-Value Operators

One recent implementation round strengthens a family that returns indices, values, or both: `torch.take_along_dim`, `argsort`, `sort`, `topk`, `kthvalue`, `argmax`, and `argmin`. These operators are small but security-critical for a static shape tool because the index tensors often feed a later gather. Losing the fact that an unpacked `indices` output is `int64`, or treating `topk` as shape-preserving, can hide a real downstream error.

TensorGuard now models the PyTorch contracts directly. `take_along_dim` returns the index shape when a dimension is supplied, checks same-rank and non-dimension broadcastability, rejects non-`int64` indices, and flattens to the index numel when `dim=None`. `argsort`, `argmax`, and `argmin` produce `int64` outputs with the same or reduced shape according to `keepdim`. `sort`, `topk`, and `kthvalue` propagate value tensors with the input dtype and index tensors with `int64`, including FX `getitem`, namedtuple `.values`/`.indices`, tensor-method calls, functional calls, and source-level tuple unpacking. Static `k` errors are refuted before runtime. The tests run real PyTorch tensors and compare both successful shapes and runtime exceptions.

The same principle now covers the gather/scatter family. `gather`, `scatter`/`scatter_add`, and `index_select` distinguish three cases that ordinary shape tools conflate: shape-invalid index tensors, shape-valid but value-invalid index tensors, and value-dependent indices whose contents are unknown statically. Stable FX-visible index buffers carry deterministic integer min/max ranges, so the verifier can report a separate high-confidence `index_bounds` bug when a real index is negative or exceeds `input.size(dim)`. Random FX-folded constants are deliberately not trusted for value bounds, and PyTorch's real dtype rule is followed exactly: these operators accept `int32` or `int64` indices, while floating or other integer widths are dtype errors.

3.2 Padding as a Library-Exact Boundary Rule

Padding looks simple until the exact PyTorch boundary conditions matter. The repository now models `F.pad` and `nn.*Pad*d` modules for `constant`, `reflect`, `replicate`, and `circular` modes. The transfer keeps PyTorch's last-dimension-backward tuple convention, preserves tuple lengths exactly as supplied, expands integer module padding according to the concrete module class, and handles negative padding as cropping. It also enforces the mode-specific runtime constraints: constant mode may pad any rank as long as the pad tuple fits the tensor rank; reflect/replicate/circular support only the documented 1D/2D/3D spatial cases; reflect padding must be smaller than the corresponding concrete input extent; circular padding may not wrap a dimension more than once; and replication is allowed to pad farther than the input size when eager PyTorch does.

The tests are differential, not merely algebraic. They execute real PyTorch for successful output shapes and for runtime errors, then require source and FX TensorGuard paths to agree. This catches exactly the kind of edge case that a paper-only rule would likely miss: for example, `nn.ReflectionPad2d((1,2))` does *not* mean symmetric 2D padding, and a circular pad equal to a dimension is legal while one larger

than the dimension is not.

3.3 Interpolation and Upsampling

`F.interpolate` and `nn.Upsample` are shape-changing even though their parameters often look like harmless presentation choices. TensorGuard now models their literal `size` and `scale_factor` contracts across 1D/2D/3D spatial tensors: scalar sizes broadcast across all spatial axes, tuple sizes must match spatial rank, and scale factors use PyTorch's integer $\lfloor \text{input} \times \text{scale} \rfloor$ output formula, including fractional downsampling and per-axis tuple scales. Options such as `align_corners`, `mode`, `recompute_scale_factor`, and `antialias` are captured so literal runtime-invalid combinations are reported, while dynamic sizes such as `x.shape[-2:]` conservatively produce symbolic spatial dimensions instead of false positives.

3.4 Recurrent Tensor Contracts

Recurrent modules are another place where a shape-preserving approximation is wrong. TensorGuard models `nn.RNN`, `nn.GRU`, and `nn.LSTM` using PyTorch's sequence contract: `batch_first` affects only the input and output layout; hidden states keep $(D \cdot L, N, H)$-style layout; bidirectionality doubles the output feature dimension but not each returned hidden direction; and projected LSTMs use `proj_size` for `output`/`h_n` while `c_n` retains `hidden_size`. Multilayer and unbatched cases share the same contract machinery, so downstream `Linear`, reshape, and tuple unpacking checks see the exact returned tensor ranks instead of a last-dimension heuristic.

Packed sequences are treated as an explicit abstention boundary. Calls such as `pack_padded_sequence` produce opaque values; recurrent layers consuming them do not fabricate dense tensor shapes, and downstream checks remain honest until a supported dense tensor reappears. The regression tests include direct verifier checks for wrong hidden/cell/projection dimensions, a live PyTorch projected bidirectional LSTM shape comparison, and a packed-sequence false-positive guard.

3.5 Examples of Transfer Precision

Reshape and view rules reason about element products and the `-1` inference sentinel. They reject concrete product mismatches, infer the missing dimension when possible, and preserve copied symbolic dimensions through `x.size(i)` and `x.shape[i]` aliases. Broadcasting implements NumPy/PyTorch right-aligned semantics, including singleton expansion and symbolic abstention. Convolution rules compute spatial output sizes from kernel, stride, padding, dilation, groups, and transpose output padding when these parameters are static. Padding rules follow eager PyTorch across constant, reflect, replicate, and circular modes, including negative cropping and mode-specific rank/extent errors. Fold/unfold rules use the same spatial arithmetic to model batched and unbatched image-column transforms, reject empty sliding block grids, non-divisible fold channel counts, and output-size/column-count reconstruction mismatches. Normalization rules check channel or normalized shape contracts, while phase annotations make train/eval-sensitive modules diagnostic instead of invisible.

Attention rules are handled at multiple layers. SDPA verifies query/key embedding compatibility and key/value source-length compatibility while preserving the output value dimension. Multi-head attention checks packed and unpacked query/key/value forms, `kdim`/`vdim`, batch-first layout, attention masks, key-padding masks, per-head weight shapes, and valid symbolic cases. Tests replay these contracts against real PyTorch modules and functional calls.

The `einsum` rule is deliberately parser-backed rather than a rank heuristic. It models explicit and implicit outputs, ASCII label ordering, repeated-label diagonals within an operand, ellipsis blocks before or after

named axes, and output ellipses that appear in non-leading positions. The public registry transfer and the verifier use the same parser, and the Z3 encoding maps output labels after the broadcasted ellipsis positions instead of assuming that labels always start at dimension zero.

Data-dependent operators are treated honestly. `masked_select`, `nonzero`, single-argument `where`, and boolean-mask indexing can produce extents that depend on runtime tensor values. TensorGuard now first checks the static part of the contract—mask broadcastability or boolean-mask prefix shape—and then emits symbolic extents plus explicit UNKNOWN reasons instead of guessing a concrete selected length. This design is repeated throughout the codebase: useful exactness where the contract is shape-determined, conservative abstention where it is not.

The linear-algebra rules illustrate the same standard for multi-output operators. `torch.linalg.svd` and `qr` return named output tuples whose shapes depend on $\min(M, N)$, `full_matrices`, and QR mode; the `r` mode's empty `Q` output is modeled exactly. Square-matrix operators (`inv`, Cholesky, and eigendecomposition) reject only statically known rank or trailing-dimension violations. `solve` preserves PyTorch's non-obvious RHS branch: vector-shaped batched RHS tensors do not broadcast like matrix RHS tensors, and `left=False` rejects the exact-batch vector branch. Focused tests compare all of these shapes and failures against the installed PyTorch dispatcher, while unsupported linalg operators remain heuristic.

The higher-order `torch.func` rules carry that exactness into differentiable wrappers. `grad` requires a rank-0 scalar body output and preserves PyTorch's int-versus-sequence `argnums` return structure. `jacrev` and `jacfwd` map every output pytree leaf with the product shape "output plus differentiated input", `jvp` validates tangent shapes, and `vjp` validates cotangent trees while documenting the pullback gradient tuple. Symbolic tangent/cotangent equalities and value-dependent closures abstain instead of creating false contracts.

Training loss functions are modeled as multi-input contracts rather than unary shape reducers. TensorGuard distinguishes class-index and probability-target `cross_entropy`, checks NLL class targets, MSE and KL broadcasting, BCE-with-logits exact target shapes, scalar versus unreduced outputs, class weights, `pos_weight`, and eager-PyTorch dtype failures. The source and `torch.fx` frontends preserve both prediction and target operands for `nn.*Loss` modules and functional calls, so malformed training snippets are rejected before the first optimizer step.

4 Developer Experience

The repository presents TensorGuard as a tool developers can actually adopt. The CLI supports zero-flag verification on common `nn.Module` files, high- confidence CI gating, domain flags such as `-no-device-check`, and soundness modes. Diagnostics are source-mapped and explain why an operation is unsafe, not just that Z3 found a model. The `-explain` path reconstructs the inference chain from inputs through transformations to the failing step. The `-fix` path suggests mechanical repairs for supported errors such as wrong `Linear.in_features` or convolution channel counts.

Adoption surfaces include a GitHub Action, SARIF for code scanning, JSON and JUnit reporters, GitHub PR annotations, pre-commit integration, nox/tox hooks, a pytest plugin, a watch mode, Jupyter/IPython cell checks, a `@tensorguard.checked` decorator, a repository config file, framework hooks for higher-level trainers, and a public API with type stubs. The packaging story includes reproducible wheels, a pinned Z3 range, pipx usage, Docker, conda-forge metadata, and a documented security policy for untrusted model files.

The diagnostic design is intentionally evidence-oriented. A bug report can carry an expected shape, actual shape, line and column, a counterexample, the domain that produced the violation, and the chain of operations that made the state refutable. When a domain is disabled, the solver encoders for that domain emit no constraints, so the flag removes solver work instead of merely filtering the final verdict. This matters for ablation experiments and for users who need predictable CI behavior.

When `torch.compile`/Dynamo or `torch.export` cannot capture a model, TensorGuard now preserves the failure as a structured graph-break attribution instead of an opaque backend exception. The report maps the failing source site back to the verifiable-fragment taxonomy—for example data-dependent Python branching, tensor-to-scalar extraction, dynamic assertions, data-dependent iteration, opaque helper calls, or mutation/JIT/custom-autograd boundaries—and attaches the exact snippet, confidence, reason, and smallest mechanical rewrite the user should try, such as replacing tensor-value branching with `torch.cond`/`torch.where` or moving scalar extraction outside `forward`. Dynamo-to-FX fallback keeps this metadata, so successful fallback verification does not erase the explanation of why the stronger backend failed.

5 Empirical Methodology

TensorGuard ships with a broad evaluation harness. The artifact is organized around questions a reviewer or user would ask.

> Does it catch real bugs? Does it avoid false positives on clean models? Does it agree with PyTorch on operator semantics? Does it scale? Which abstract domains matter? How does it compare to baselines? Can the entire claim set be regenerated from committed scripts?

The repository answers these questions through several benchlines and experiments rather than a single benchmark number. The frozen ground-truth corpus in `real_benchmarks/` contains clean and buggy PyTorch modules with content hashes and labels. The GitHub-mined dataset under `experiments_v5/github_bug_minin` records real public issue and PR signals associated with PyTorch runtime error signatures. Fuzzing harnesses generate valid modules and mutation harnesses inject shape, device, and dtype faults. Differential conformance tests cross-check transfer functions against the live torch dispatcher. Baseline comparison scripts evaluate PyTea and runtime-tool alternatives. Dashboards and JSON artifacts track coverage, precision/recall, latency, ablations, false positives, and statistical rigor.

Evidence family	Repository artifact and purpose
Ground-truth corpus	`real_benchmarks/`, `tests/test_real_benchmarks.py`. Frozen clean/buggy modules with a manifest, hashes, labels, and loader checks.
GitHub-mined bugs	`experiments_v5/github_bug_mining/`. Offline dataset plus miner for public shape/device bug candidates from issue and PR history.
Precision/recall	`tests/test_precision_recall.py`, `tests/test_hard_recall.py`, `experiments/`. Tracks caught bugs, clean-model behavior, and high-confidence safety.
False-positive stress	`tests/test_fp_stress.py`, `tests/test_sound_mode_fp.py`. Exercises clean real-model patterns and sound-mode no-false-positive posture.
Differential dispatch	`tests/test_differential_dispatcher.py`, `tests/test_conformance_oracle.py`, `tests/test_propagator_contracts_runtime` Compares transfer outputs to live PyTorch behavior.
Fuzzing and mutation	`tests/test_diff_fuzz.py`, `tests/test_neg_fuzz.py`, `tests/test_hypothesis_module_ast.py`, `tests/test_mutation_clean_models.py`. Searches for disagreements and injected bugs.
Operator coverage	`operator_confidence_table.json`, `tests/test_operator_coverage.py`, `tests/test_operator_coverage_gate.py`, `tests/test_real_model_operator_coverage.py`. Tracks supported operators, confidence tags, release gates, and frequency-weighted real-model coverage over torchvision, timm, and HuggingFace traces.
Latency and scaling	`tests/test_cost_benchmarks.py`, `tests/test_latency_budgets.py`, `tests/test_scaling_study.py`. Measures model-size and solver-cost behavior.
Baselines	`tests/test_baselines.py`, `tests/test_baseline_head_to_head.py`, `tests/test_headtohead_n15.py`. Compares against PyTea, runtime checks, and type-oriented alternatives where applicable.
Ablations	`tests/test_domain_ablation.py`, `tests/test_reduced_product_ablation.py`, `tests/test_cegar_depth_ablation.py`, `tests/test_verified_reduction_diff.py`. Measures contribution of domains and refinement depth.
Dashboards	`tests/test_dashboard.py`, `tests/test_build_dashboard.py`, `tests/test_leaderboard.py`. Regenerates static result views for reviewers and contributors.
Statistical rigor	`tests/test_statistical_power.py`, `tests/test_significance.py`, `tests/test_corrected_meta_analysis.py`. Documents effect sizes, power, and aggregate evidence.

6 Evaluation Highlights

The strongest evidence is not one number; it is the agreement between many small, independently checkable claims. TensorGuard's benchmark suite exercises real failure modes: wrong `Linear` feature counts after flattening, malformed convolution channels, bad Q/K/V dimensions, attention mask errors, invalid reshape products, device mismatches between buffers and inputs, dtype divergences, gradient detach bugs, invalid embedding-bag offsets and per-sample-weight contracts, invalid sparse compressed layouts and sparse-dense kernels, named tensor alignment failures, incompatible loss targets, incompatible distribution batch/event shapes, export contracts, quantization observer calibration/qscheme and quant/dequant placement, ONNX shape-inference disagreements, mixed-precision/autocast dtype drift, FSDP2/DTensor local-shard mistakes, and tensor-parallel sharding mistakes. Checkpoint-schema gates add another resume-only failure class: model `state_dict` loads are checked for missing or unexpected keys, tied weights whose aliases disagree, adapter-only LoRA matrices, tensor-parallel shard coverage, and dtype drift that PyTorch would otherwise silently cast. The dedicated LoRA/PEFT gate checks HuggingFace-style adapter keys and live modules for rank, target-module matching, merged/unmerged state, and quantized-base compatibility. Optimizer-state checks cover AdamW and fused-AdamW moment buffers, Adafactor factored `row_var`/`col_var` buffers, lazy pre-step state, and declared ZeRO-style shards before an incompatible checkpoint can corrupt a resumed run.

The conformance tests are especially important. A static rule is only useful if it matches the runtime library. The repository therefore includes tests that construct real tensors, run the corresponding PyTorch operator, and compare the resulting shape or runtime exception to TensorGuard's predicted contract. This pattern appears for cat, einsum, reshape, broadcasting, convolution, attention, recurrent modules, fold/unfold, padding, grid sampling, embedding-bag pooling and jagged-offset contracts, sparse layouts and sparse operators, complex FFT views, named tensors, distributions, vmap, loss functions, chunk/split partitioning, and index-value operators. When PyTorch's edge-case behavior is surprising, as with `take_along_dim(dim=None)` flattening before selection or `topk` rejecting oversized `k`, the static rule follows PyTorch rather than a simplified textbook semantics.

The ablation suite makes the multi-domain design measurable. Shape alone catches the most common architectural errors, but device and gradient domains catch bugs that shape-only tools miss. Dtype reasoning catches mixed-precision and cast-related failures. Phase information explains train/eval-sensitive patterns. The reduced product is valuable because real failures often cross domain boundaries: a compile/export path may require a shape, dtype, device, and layout combination, not merely one axis of correctness.

Latency and scalability scripts ensure the verifier remains useful. Solver queries are cached or avoided when a transfer can be decided syntactically. Constraint simplification performs constant folding, common-subexpression sharing, divisibility normalization, and short-circuiting. Per-module verification can run in parallel, and incremental analysis reuses results when only part of a model changes. Timeout and anytime modes return sound partial results instead of claiming full coverage after a budget expires.

Operator coverage is measured by the operators that actually occur in traced models, not just by the raw public PyTorch surface. The latest census traces torchvision, timm, and no-download HuggingFace models, publishes a before/after matrix, and pins each newly counted hot operator with real extractor tests: `torchvision.ops.stochastic_depth` is treated as shape-preserving, functional `layer_norm` and `adaptive_avg_pool2d` build synthetic FX layers, and SDPA maps to the existing attention contract. Scalar shape-metadata queries such as `size`, `dim`, and integer `floordiv` are reported separately instead of being counted as tensor transfer functions.

7 Formal and Trust Story

TensorGuard does not ask users to trust only implementation tests. The repository contains Lean artifacts for several core rules and proof obligations, including CEGAR convergence bounds, infeasible-contract abstention, fragment modes, device/dtype transfer soundness, SMT encoding faithfulness, cross-domain composition, and operator-specific rules such as cat and reshape families. The Lean audit checks theorem statements for sorry-free status under the trusted kernel axioms used by the environment.

The formalization is intentionally scoped. It does not claim to prove all of PyTorch. Instead, it proves the algebraic cores that the implementation relies on and pairs them with runtime conformance tests for library behavior. For a research artifact, this is a practical trust split: Lean validates the abstract rule, Z3 discharges symbolic conditions in the implementation, and PyTorch oracle tests validate that the rule corresponds to the library release being tested.

The soundness contract documents the remaining boundary. Unsupported or heuristic operators are not silently certified. Opaque layers and data-dependent control flow produce UNKNOWN in strict modes. Symbolic sizes are preserved instead of guessed. This is visible in the code. For chunk/split, for example, concrete dimensions produce exact per-output sizes and concrete unpack-count errors; symbolic split dimensions produce fresh symbolic split axes, preventing false claims about a final chunk size.

8 Implementation Surfaces

The system is intentionally broad because tensor bugs are not confined to a single frontend. The repository includes modules for source analysis, model checking, graph compilation, torch integration, compile-guard comparison, export extraction, Dynamo gap analysis, framework hooks, sparse and complex verification, SDPA and MHA verification, distribution checks, named tensor checks, vmap checks, quant/export checks, distributed verification, LoRA verification, training-loop checks, reporters, SARIF, baselines, dashboards, proof certificates, and API stability.

Surface	Representative capability
`src/api.py`	Public `analyze`, `verify_architecture`, and `verify_module` entry points with soundness modes and domain flags.
`src/model_checker.py`	AST graph extraction, symbolic state propagation, Z3 safety checks, device, phase, gradient, dtype, recurrent, and shape transitions.
`src/tensor_shapes.py`	Shape algebra, transfer helpers, AST shape analyzer, reshape/broadcast/cat and chunk/split shape computation.
`src/fx_extractor.py` and `src/export_extractor.py`	Tracing and export-oriented frontends for real modules and deployment flows.
`src/torch_integration.py`	Guarded compile/export/package pre-passes that surface TensorGuard violations before compiler or exporter failures, then validate ONNX checker/opset and shape-inference round trips on emitted artifacts.
`src/mha_verify.py`, `src/sdpa_verify.py`	Attention-family contracts for q/k/v, masks, head counts, and output shapes.
`src/embedding_bag_verify.py`	PyTorch and TorchRec embedding-bag contracts for offsets, per-sample weights, pooling modes, index bounds, and jagged/ragged abstention.
`src/sparse_verify.py`, `src/complex_verify.py`	Sparse layout, sparse-dense operator, conversion, and complex/FFT shape-dtype contracts.
`src/distributions_verify.py`, `src/named_tensor_verify.py`, `src/vmap_verify.py`, `src/func_autodiff_verify.py`, `src/loss_verify.py`, `src/torchvision_v2_verify.py`	High-value library contracts beyond ordinary dense layers, including image preprocessing transforms whose tensor shapes feed the model.
`src/quant_export_checks.py`, `src/mixed_precision_verify.py`, `src/gguf_export.py`, `src/checkpoint_verify.py`, `src/optimizer_state_verify.py`, `src/distributed_verification.py`, `src/tensor_parallel_checks.py`, `src/lora_verification.py`, `src/cuda_graph_capture.py`	Deployment, mixed-precision, distributed training, and adapter-compatibility gates, including GGUF/llama.cpp checkpoint preflight checks, model `state_dict` schema checks, AdamW/Adafactor optimizer-state resume checks, and FSDP2/DTensor rank-local shard contracts checked against PyTorch's own placement splitter, explicit pipeline stage-boundary contracts, and MQA/GQA tensor-parallel attention contracts, plus CUDA graph capture diagnostics for dynamic allocation, data-dependent shapes, unsupported operations, and static input replay.
`src/reporters.py`, `src/sarif_codescan.py`, `src/github_action.py`	Machine-readable and platform-native reporting.
`lean/TensorGuard/*.lean`	Machine-checked statements for selected transfer rules, mode boundaries, encoding faithfulness, and CEGAR reasoning.

9 Artifact Reproducibility

The artifact is designed to be regenerated. The `make reproduce` harness and reproducibility scripts update headline results, numeric claim audits, benchmark deltas, scaling curves, CEGAR measurements, and paper evidence indices. Tests pin committed JSON artifacts against freshly generated outputs. The benchmark corpus has content hashes, datasheets, citation metadata, and loader checks. The operator confidence table is generated from code. The dashboard is regenerated from committed results. The public docs explain what the verifier can and cannot prove.

Reproducibility is paired with hygiene. Third-party source trees are quarantined or removed unless license-compatible. The security policy and threat model acknowledge that untrusted model files are parsed and should not be executed dynamically. Safe loading paths use AST/fx-style extraction rather than arbitrary imports where possible. Baselines and suppressions let teams adopt the tool incrementally without hiding new regressions.

10 Limitations

TensorGuard is intentionally conservative. It is not a complete theorem prover for all Python or all PyTorch. It handles a documented verifiable fragment and falls back to UNKNOWN outside that fragment in strict modes. Data-dependent shape operators, arbitrary Python side effects, dynamic module mutation, runtime-generated code, and library internals that cannot be traced or modeled may require abstention. Symbolic arithmetic is limited to decidable fragments and practical solver budgets. Some integrations verify preconditions for export, compile, serving, quantization, or distributed execution rather than replacing those systems' own validators.

These limitations are design choices, not hidden caveats. The repository keeps an honest limitations document, a formal fragment specification, confidence tags for operators, and tests that assert unsupported constructs become UNKNOWN rather than SAFE. The most important invariant is not that every program is proved; it is that a strict safe verdict means what it says.

11 Why the Artifact Matters

For a tool to be credible at PLDI, OOPSLA, ISSTA, or a systems-for-ML venue, it must connect a formal idea to the messiness of real code. TensorGuard does that in three ways. First, it makes refinement-style tensor verification usable from normal PyTorch files with zero annotations. Second, it exposes enough product surface–CLI, CI, SARIF, Jupyter, decorators, framework hooks, export gates, Docker, conda, pipx, docs–that adoption can be evaluated rather than imagined. Third, it ships an artifact that reviewers can run, audit, and extend.

The codebase also provides a platform for future work. New operator rules can be added with confidence tags, conformance tests, and proof footprints. New benchmarks can enter the frozen corpus through provenance and hashing. New frontends can lower into the same graph representation. New proof-carrying certificates can reuse the existing safety certificate and replay infrastructure. The next natural milestones are larger blind real-bug corpora, broader deployment galleries, independently checkable certificates, and a community leaderboard that makes tensor-verifier progress measurable.

12 Conclusion

TensorGuard demonstrates that static verification for PyTorch tensor programs can be both principled and practical. The repository combines a refinement-type view of tensors, a multi-domain abstract state, Z3-backed checks, CEGAR refinement, soundness modes, precise operator transfer functions, real frontends, formal artifacts, and an unusually broad reproducibility harness. The latest gather/scatter and padding work illustrates the project's standard: match PyTorch's exact edge-case semantics, propagate precision to downstream model checks, abstain on symbolic or value-dependent uncertainty, and pin the behavior with live-library tests.

The result is a verifier that catches high-value architecture, device, dtype, phase, gradient, export, and deployment bugs before expensive training or opaque compiler failures. It is a research artifact with a clear trust boundary and a developer tool with practical adoption paths–the combination needed for a best-paper-quality static analysis system in modern ML software.

A Capability Matrix

Capability	What the repository demonstrates
Zero-annotation source verification	Inference from constructors, layer definitions, tensor factories, asserts, shape aliases, and dataflow without requiring user type annotations.
Shape algebra	Matmul, reshape/view, flatten, broadcasting, cat, stack-family aliases, chunk, split, padding, transpose, permute, squeeze, unsqueeze, reductions, indexing, convolution, pooling, fold/unfold, embedding, and attention rules.
Device reasoning	Propagation through tensor creation, module calls, `to`/`cuda`/ `cpu`, buffers, and cross-device operations.
Dtype reasoning	Promotion/cast checks, explicit autocast backend/dtype contexts, AMP allowlists, GradScaler requirements, parameter/input dtype matching, quantization/export dtype gates, and dtype-related runtime-error prevention.
Phase reasoning	Train/eval tagging and phase-sensitive diagnostics for modules such as BatchNorm and Dropout.
Gradient reasoning	Detection of detached or broken gradient flow and training-loop checks around optimizers, parameters, and checkpoint-like patterns.
Training losses	Target-shape, reduction, dtype, class-weight, and `pos_weight` contracts for cross entropy, NLL, MSE, BCE-with-logits, and KL losses.
Stride and permutation	Layout and viewability support that keeps reshape/permute reasoning from collapsing to rank-only approximations.
Attention	SDPA and MHA contracts, q/k/v dimensions, kdim/vdim, masks, key padding masks, batch-first layout, grouped head variants, MQA/GQA tensor-parallel partitioning, KV-head replication, and sequence-parallel LayerNorm safety.
Embedding bags	`nn.EmbeddingBag`, `F.embedding_bag`, table dimensions, offsets, `include_last_offset`, per-sample weights, pooling modes, index bounds, and TorchRec-style jagged abstention.
Sparse tensors	COO/CSR/CSC/BSR/BSC shape and blocksize invariants, sparse-dense `mm`/`addmm`, CSR `sampled_addmm`, sparse softmax, coalesce, `to_dense`, and layout conversions.
Torchvision v2 transforms	Source-level tensor contracts for resize, crops, padding, normalization, flips/color/dtype transforms, and PIL-only abstention.
Named tensors	`refine_names` and `align_to` alignment checks.
Complex and FFT	`view_as_real`, `view_as_complex`, FFT shape/dtype contracts.
Distributions	Batch shape, event shape, sample shape, and log-probability compatibility for probabilistic code.
vmap/functorch	Batched dimension transfer and out-dim contract checks.

Export and compile	Pre-pass verification for `torch.compile`, `torch.export`, ONNX, AOTInductor packaging, and deployment gates, including package-time layout, dtype, device, dynamic-guard, ONNX opset availability, lowering checks, and post-export ONNX shape-inference agreement. The compile-guard audit compares TensorGuard rank, concrete-dimension, repeated-symbol, and layer-boundary constraints against Dynamo `ExplainOutput.out_guards`, reporting missing shape-env guards rather than trusting a compiled graph by inspection. GGUF/llama.cpp export gates check q/k/v packing, rotary dimensions, vocab projections, quant block divisibility, and metadata consistency before an external converter is invoked; model checkpoint gates check missing/unexpected keys, tied weights, adapter-only LoRA matrices, tensor-parallel shards, and silent dtype drift; LoRA/PEFT gates check rank, target-module matching, merge state, and quantized bases; optimizer-state gates check AdamW, Adafactor, fused AdamW, and sharded resume buffers before `load_state_dict`; serving-schema gates check request fields, preprocessing tensor layouts, model outputs, and response payloads before a TorchServe/FastAPI wrapper turns a bad batch into an opaque inference failure; CUDA graph gates check allocation-free captured regions, value-independent output shapes, unsupported host-synchronizing operations, and replay-stable input shape/dtype/device/stride contracts before capture.
Distributed and adapters	DDP/FSDP/FSDP2/DTensor/per-parameter, explicit pipeline-boundary, and tensor-parallel checks, plus optimizer-state, checkpoint-schema, and LoRA/PEFT adapter compatibility gates.
Reports and integrations	CLI, JSON, JUnit, SARIF, GitHub annotations, GitHub Action, pre-commit, pytest plugin, nox/tox, watch mode, Jupyter, decorators, and framework hooks.
Formal artifacts	Lean proof files, axiom audits, SMT encoding faithfulness checks, and proof certificate infrastructure.
Reproducibility	Frozen corpora, generated dashboards, numeric claim audits, benchmark scripts, hash manifests, datasheets, artifact badges, Docker/conda/source modes.

B Benchline and Experiment Inventory

Artifact	**Role in the evaluation story**
`tests/test_stack_family.py`	Live PyTorch conformance for `torch.stack`, `hstack`, `vstack`, `dstack`, `column_stack`, and `row_stack`: singleton TensorLists, negative dims, scalar/1D/2D/3D rank promotions, source/FX extraction, and static error parity.
`tests/test_split_chunk_precision.py`	Live PyTorch cross-checks and verifier regressions for exact chunk/split partition rules, empty chunks, negative dims, function/method forms, and downstream Linear/cat behavior.
`tests/test_mha_verify.py`	Multi-head attention conformance against real `nn.MultiheadAttention` and functional attention calls.

tests/test_einsum_precise.py	Equation parser and output-shape tests for einsum, including repeated-label diagonals, ellipsis placement, ASCII output ordering, registry-transfer parity, and Z3 output-position constraints.
tests/test_index_value_ops_precise.py, tests/test_gather_scatter_index_bounds.py	Live PyTorch conformance for `take_along_dim`, `argsort`, `sort`, `topk`, `kthvalue`, `argmax`, and `argmin`, plus gather/scatter/index-select index dtype and static value-bound errors.
tests/test_reshape_precise.py, tests/test_reshape_divisibility_opts.py	Product reasoning, `-1` inference, and optimized divisibility constraints for reshapes/views.
tests/test_broadcast_expand_precise.py, tests/test_broadcast_correspondence.py	Precise broadcast and expand semantics, including singleton expansion and dim correspondence.
tests/test_conv1d_rule.py, tests/test_conv2d_rule.py, tests/test_conv3d_rule.py, tests/test_pool2d_rule.py, tests/test_convtranspose_rule.py	Convolution and pooling transfer functions over spatial arithmetic.
tests/test_unfold_fold_spatial.py	Live PyTorch conformance for `nn.Unfold`, `nn.Fold`, `F.unfold`, and `F.fold`, including batched/unbatched ranks, dilation/padding/stride block counts, empty grids, non-divisible fold channels, and impossible fold column-count/output-size reconstructions.
tests/test_pad_semantics.py	Live PyTorch conformance for `F.pad` and `nn.*Pad*d`: exact constant/reflect/replicate/circular output shapes, source and FX parity, negative cropping, non-reinterpreted tuple lengths, 3D circular modules, and runtime-error parity for invalid ranks, reflect extents, and circular over-wraps.
tests/test_interpolate_precise.py	Live PyTorch conformance for `F.interpolate` and `nn.Upsample`: literal `size` vs. `scale_factor`, fractional and tuple scales, rank-3/4/5 formulas, source and FX parity, dynamic-size abstention, downstream shape flow, and runtime-error parity for invalid modes/options.
tests/test_dtype_precise.py, tests/test_dtype_promote_chain.py, tests/test_bool_dtype_soundness.py, tests/test_mixed_precision_verify.py	Dtype propagation, promotion, boolean/integer safety, and live PyTorch CPU-autocast parity for mixed-precision policy gates.
tests/test_loss_verify.py	Loss-function conformance for target shape, reduction, dtype, class weights, `pos_weight`, source snippets, functional calls, FX modules, and eager PyTorch runtime-error parity.
tests/test_torchvision_v2_verify.py	Live torchvision v2 conformance for resize/crop/pad/normalize/flips, source `Compose` and inline constructor extraction, runtime-error parity, and PIL-only symbolic abstention.
tests/test_device_propagation_precise.py, tests/test_device_placement_transfer.py	Device movement and cross-device error detection.
tests/test_grad_flow_transfer.py, tests/test_training_loop_checks.py	Gradient and training-loop contracts beyond forward shape checking.

tests/test_onnx_export_gate.py, tests/test_onnx_opset_gate.py, tests/test_export_aot_gate.py, tests/test_gguf_export_gate.py, tests/test_checkpoint_verify.py, tests/test_optimizer_state_verify.py, tests/test_serving_schema.py, tests/test_cuda_graph_capture.py, tests/test_quant_export_checks.py, tests/test_quantization_verify.py, tests/test_deployment_budgets.py, tests/test_deployment_gallery.py	Deployment-facing gates for export, `torch.export.Dim` ranges and relations, ONNX opset-version availability and dynamic-shape support, AOT package input/layout/dtype/device/lowering contracts, and source/eager/FX quantization, plus GGUF/llama.cpp checkpoint checks over real transformer state_dict tensors, logical quantized descriptors, model checkpoint-schema compatibility for tied weights, LoRA adapters, tensor-parallel shards, and dtype drift, plus real optimizer state_dict compatibility for AdamW, Adafactor, fused AdamW, lazy state, sharded resume buffers, and load-time shape rejection; FastAPI/TorchServe-style serving pipelines whose request, preprocessing-output, model-output, and response contracts are checked against real PyTorch modules, including an NHWC-vs-NCHW preprocessing bug rejected before `forward`, and CUDA graph eligibility checks that run real modules while flagging dynamic allocation, data-dependent shape operators, unsupported host synchronization, and replay input shape/stride drift.
tests/test_distributed_verification.py, tests/test_tensor_parallel_checks.py	FSDP, DeepSpeed, FSDP2, DTensor, per-parameter, pipeline-schedule, and tensor-parallel shape/device contracts, including rank-specific shard-shape conformance against PyTorch's `torch.distributed.tensor` placement helper and MQA/GQA attention conformance against real PyTorch sharded executions plus installed HuggingFace `LlamaAttention` projection shapes.
tests/test_lora_verification.py	Adapter rank and target-module compatibility checks.
tests/test_security.py, tests/test_threats_to_validity.py	Security policy and evaluation-threat documentation.
tests/test_docs_getting_started.py, tests/test_docs_limitations.py	Executable documentation checks for onboarding and limitations.
tests/test_paper_evidence_index.py, tests/test_paper_capsule_wiring.py	Paper/artifact wiring so claims can be tied to scripts and committed evidence.

C Transfer Rule Ledger

This appendix summarizes representative transfer rules as contracts rather than as an operator checklist. The purpose is to make clear which parts of the system are exact, which are sound but conservative, and which intentionally abstain.

Family	Static contract	Evidence path
Tensor factories	`zeros`, `ones`, `rand`, `randn`, `empty`, `full`, and integer factories produce shapes determined by static size arguments, independent of random values.	Factory transfer tests and RNG shape tests; output shapes are propagated even when tensor contents are unknown.

Linear layers	The last input dimension must equal `in_features`; output replaces the last dimension with `out_features`.	Unit tests for clean and buggy linear models, autofix tests, and end-to-end module verification.
Recurrent modules	`RNN`, `GRU`, and `LSTM` check input feature size, preserve sequence/batch layout, account for layers and directions, distinguish LSTM `h_n` from `c_n`, support projections, and abstain on packed sequences.	Focused recurrent tests plus a live PyTorch projected bidirectional LSTM shape oracle and packed-sequence abstention regression.
Matrix multiplication	Contracting dimensions must match; batched prefixes are propagated or broadcast where supported.	Shape arithmetic tests, Z3 symbolic checks, model-checker violations, and baseline bug cases.
Convolution	Input rank, channels, groups, kernel, stride, padding, dilation, and output padding determine channel and spatial output shapes.	Conv1d/2d/3d, transpose convolution, pooling, and pixel-shuffle regression tests.
Fold/unfold	`unfold` maps image tensors to image-column tensors with exact kernel/dilation/padding/stride block counts; `fold` inverts the shape only when channel divisibility and the output-size-implied sliding-block product match the input columns.	Live PyTorch differential tests for module and functional forms, including runtime-error parity for empty grids and impossible reconstruction counts.
Padding	`F.pad` and `nn.*Pad*d` apply tuple pairs from the last dimension backward; constant mode accepts any compatible rank, while reflect, replicate, and circular modes enforce PyTorch's spatial-rank cases and concrete extent constraints. Negative padding crops where PyTorch permits it.	Live PyTorch differential tests for functional and module forms across source and FX, including invalid-rank, reflect-bound, circular-wrap, and large replicate-padding cases.
Interpolation	`F.interpolate` and `nn.Upsample` preserve batch/channel axes and compute spatial axes from literal `size` or `scale_factor`; dynamic size specs abstain soundly.	Live PyTorch differential tests for fractional scales, tuple scales, 1D/2D/3D spatial ranks, source/FX parity, invalid option combinations, and downstream linear-shape checking.
Reshape/view	Element product is preserved; `-1` is inferred when uniquely determined; copied dimensions from `x.size(i)` and `x.shape[i]` retain aliases.	Precise reshape tests, divisibility optimization tests, Lean-adjacent proof footprints, and real model regressions.

Flatten/unflatten	Flatten multiplies a contiguous dimension range when concrete and otherwise preserves rank with a symbolic product; unflatten checks product compatibility.	Flatten, unflatten, and reshape-chain tests.
Broadcast and expand	Right-aligned dimensions must be equal or singleton; expand may only grow singleton dimensions, and `-1` preserves an existing dimension where PyTorch allows it.	Broadcast correspondence, expand precision, and elementwise-operation tests.
Cat	All non-concatenation dimensions must match; concatenation dimension sums concrete sizes or becomes symbolic. Negative dims are normalized.	Lean cat rule, live PyTorch cat tests, analyzer and model-checker cat regressions.
Stack	Base `stack` requires equal ranks and static-equal concrete dimensions, then inserts a new axis; `hstack`/`vstack`/`dstack`/`column_stack`/`row_stack` apply PyTorch's exact scalar, 1D, 2D, and 3D rank promotions before concatenating along the library-defined axis.	Live PyTorch differential tests in `tests/test_stack_family.py`, including source and FX extraction plus runtime-error parity.
Chunk/split	Exact PyTorch partition sizes for concrete axes; concrete unpack-count errors; fresh-symbolic split axis when output count or size depends on symbols.	Live torch 2.9.1 cross-checks in `tests/test_split_chunk_precise.py` and downstream Linear/cat model checks.
Squeeze/unsqueeze	Squeeze removes singleton axes when statically known; unsqueeze inserts a singleton axis with negative-dim normalization.	Tensor-shape analyzer tests and model-checker step propagation.
Transpose/permute	Axis swaps and permutations reorder shape dimensions; identity permutes can be diagnosed as suspicious no-ops.	Transpose tests, permutation theory tests, identity-permute diagnostics.
Reductions	Dim reductions remove an axis or replace it with 1 under `keepdim`; global reductions produce scalar shapes.	Reduction tests for mean, sum, min, max, variance, standard deviation, and arg reductions.

Indexing and gather	Gather and `take_along_dim` return index-derived shapes (`dim=None` flattens); index-select replaces the selected dimension; scatter preserves the input shape; narrow replaces a dimension with a known length; sort/topk/kthvalue preserve value dtypes while producing `int64` indices; stable gather/scatter/index-select index buffers check dtype and static bounds while value-dependent indices abstain.	Indexing/gather tests, index-value precise tests, gather/scatter bounds tests, and extended operator tests.
Masked and value-dependent ops	Masked select, nonzero, one-argument where, and boolean masks check static mask contracts, then use symbolic selected lengths and explicit unknown reasons for value-dependent extents.	Boolean-mask tests against live PyTorch plus soundness-boundary tests for data-dependent ranks.
Einsum	Equation parser handles explicit and implicit outputs, ellipses, broadcasting, ASCII output order, and repeated-label diagonal constraints where supported.	Einsum theory tests, precise einsum tests, and randomized torch differential batteries.
Attention	Q/K/V ranks, embedding dimensions, source lengths, batch layout, masks, head-related contracts, MQA/GQA tensor-parallel head partitioning, KV-head replication, and sequence-parallel LayerNorm axes are checked.	MHA and SDPA tests against real PyTorch modules and functional calls, plus tensor-parallel attention tests over real PyTorch sharded executions and installed HuggingFace `LlamaAttention` projection shapes.
Normalization	BatchNorm, LayerNorm, GroupNorm, and InstanceNorm preserve shapes while checking channel or normalized-shape constraints when static.	Normalization rule tests and phase-flow tests.
Sparse layouts	Compressed and coordinate layouts carry index/value shape invariants and blocksize constraints.	Sparse verification tests across COO, CSR, CSC, BSR, and BSC.
Complex and FFT	Complex view conversions enforce final-dimension and dtype contracts; FFT families compute output dimensions such as real FFT half-spectrum sizes.	Complex verification tests, FFT tests, and operator confidence tags.

Linear algebra	`solve`, inverse, Cholesky, SVD, eigendecomposition, and QR enforce rank, square-matrix, batch-broadcast, mode, and named output-tuple contracts.	Linalg differential tests against real PyTorch plus sound per-op confidence tags.
Named tensors	Name refinement and alignment require axis-name consistency and legal reorder operations.	Named tensor verification tests.
Distribution shapes	Batch, event, sample, and log-probability shapes must be mutually compatible.	Distribution verification tests and probabilistic-model examples.
vmap/functorch and `torch.func` autodiff	Batched dimensions are inserted, removed, or moved according to `in_dims` and `out_dims`; autodiff wrappers produce gradient/Jacobian/JVP/VJP pytrees with PyTorch-faithful `argnums`, tangent, and cotangent shape contracts; impossible concrete contracts are rejected.	vmap verification tests plus `tests/test_func_autodiff_verify.py` live dispatcher checks.

Export gates	Export, ONNX, AOTInductor, GGUF, CUDA graph, serving, quantization, and mixed-precision paths check static preconditions before invoking downstream compilers; dynamic `torch.export.Dim` ranges are reconciled with TensorGuard input-shape symbols before tracing; ONNX exports check exporter opset bounds, lowered ATen minimum opsets, and backend-specific dynamic-shape support before writing a proto; and AOT packages additionally reject invalid example layouts, devices, dtypes, missing dynamic guards, and denied ATen lowerings before the packager writes an artifact. Compile-guard comparison normalizes Dynamo rank, shape-index, range, and cross-input equality guards and checks them against TensorGuard symbolic constraints on real `nn.Linear`, residual, and convolution modules. The GGUF gate checks architecture-prefixed llama.cpp metadata, q/k/v and packed-qkv tensor shapes, grouped-query head consistency, rotary dimensions, vocab/output projection padding, and GGUF quant block sizes before the converter writes a checkpoint; the optimizer-state gate checks AdamW/fused-AdamW moment shape, Adafactor factored variance buffers, lazy initialization, sharded-state coverage, and checkpoint resume compatibility before `optimizer.load_state_dict`; serving gates validate captured request fields, preprocessed tensor shapes/dtypes/devices, model-output contracts, and postprocessed response payloads before invoking or returning from FastAPI/TorchServe-style wrappers; the CUDA graph gate flags fresh allocations	ONNX, AOT, GGUF, CUDA graph, serving-schema, quantization, mixed-precision, compile-guard, and guarded compile/export tests.

Distributed and adapter gates	Sharding, explicit pipeline stage boundaries, tensor-parallel linears and MQA/GQA attention, optimizer state, checkpoints, and LoRA adapters have shape/dtype/rank contracts layered over base verification; PEFT-style `target_modules`, merged/unmerged flags, and quantized bases are checked without mutating the model.	Distributed verification, tensor-parallel, optimizer-state, training-loop, checkpoint, and LoRA tests, including real PyTorch optimizer state dicts, real PyTorch sharded attention, and HuggingFace `LlamaAttention` projection checks.

D Experiment Protocol Notes

The empirical suite is intentionally redundant. Each protocol checks a different failure mode in the claim stack, and overlap is useful: a bug caught by a real-bug corpus, a mutation benchmark, and a live torch conformance test is less likely to be an artifact of one benchmark's construction.

D.1 Real-bug protocol

Real bugs are valuable only when provenance is preserved. The repository's corpus entries carry labels, hashes, and loader checks so that a benchmark case cannot silently drift. GitHub-mined cases are derived from public issue and PR signals, especially runtime error messages that indicate shape or device failures. The intent is not to treat every mined candidate as a theorem; it is to create a large, auditable pool that can be stratified, manually labeled, filtered into blind splits, and reused by future tools.

D.2 Clean-model protocol

False positives are more damaging than missed bugs for a CI gate. Clean-model tests therefore exercise patterns that static tools often mishandle: symbolic batch sizes, shape aliases, helper functions, container modules, attention wrappers, non-divisible but valid partitions, singleton broadcasting, empty sections, and deployment preconditions that should abstain rather than refute. The strict soundness mode is evaluated on these clean cases to maintain the promise that a trusted safe verdict is not an accident of permissive defaults.

D.3 Differential protocol

For operator semantics, PyTorch is the executable specification. Differential tests generate tensors, run PyTorch, and compare the observed shape or exception to TensorGuard's transfer function. This protocol is especially useful for edge cases whose behavior is not obvious from memory: `torch.chunk` returning fewer than requested chunks, zero-size split sections, broadcasting of attention masks, padding tuple/rank corner cases, compressed sparse block constraints, and named-axis alignment. The same oracle style now covers embedding-bag corner cases such as 1D inputs requiring offsets, 2D inputs forbidding offsets, `include_last_offset` changing bag count, and per-sample weights being legal only in `mode="sum"`. It also covers torchvision v2 image transforms, where `CenterCrop` accepts rank-2 tensors, `Resize` and `Pad` require tensor-image rank on the tested runtime, and `Normalize` can broadcast a singleton channel

dimension. Recent linalg tests add another such case: QR's `mode="r"` empty `Q`, SVD full-vs-reduced output tuples, and `solve`'s vector/matrix RHS disambiguation are all dispatcher-visible but easy to misremember.

D.4 Mutation protocol

Mutation tests start from clean models and introduce controlled faults: wrong feature counts, wrong channels, bad reshape products, device moves, dtype casts, gradient detaches, invalid embedding-bag offsets/weights, invalid sparse metadata, and export-incompatible layouts. A mutation is useful only if it represents a runtime or semantic failure that real code could contain. The harness therefore records the mutant, the expected outcome, and the evidence that the mutation is admitted by the source syntax but rejected by the verifier or runtime.

D.5 Ablation protocol

Ablations isolate the contribution of each abstract domain and each refinement component. Shape-only, device-only, dtype-only, gradient-only, independent domains, reduced products, CEGAR depths, operator coverage levels, solver timeouts, and frontend variants can be compared. This is the right protocol for answering whether the tool's complexity is justified. If a domain never catches a unique bug, it should either be simplified or documented as diagnostic.

D.6 Latency and deployment budget protocol

Static verification must run before it can help. The latency protocol measures small, medium, and large models; import-time, install-time, and analysis-time costs; solver calls avoided by syntactic transfer functions; incremental reverification under diffs; parallel per-module verification; and timeout behavior. The important metric is not only mean runtime but the tail behavior under symbolic constraints and unsupported regions.

Deployment adds a second budget surface: the verifier must be cheap enough to run both before and after graph capture. The deployment budget harness measures real FX pre-export/pre-compile checks, `torch.export` post-export checks, and `torch.dynamo` post-compile checks when the interpreter supports `torch.compile`; unsupported compile backends become explicit skips rather than false passes. Its committed artifact stores deterministic latency/memory budgets plus non-dominated per-backend Pareto curves, while the live gate measures wall-clock time and verifier-stage memory on real `nn.Module`s. A companion deployment gallery stresses architectural diversity instead of cost: compact ResNet, ViT, Llama-style, diffusion U-Net, recommender, and speech blocks execute eagerly, then pass both the FX pre-export gate and the `torch.export` post-capture gate in supported environments.

Protocol	**Primary risk controlled**	**Representative assertion**
Real-bug corpus	Overfitting to toy bugs	Frozen labels and hashes reproduce expected verdicts.
Clean-model stress	CI-hostile false positives	Strict mode does not refute valid real-model patterns.
Differential torch oracle	Incorrect operator semantics	Static transfer matches PyTorch shape or exception on generated cases.
Mutation benchmark	Weak recall on realistic edits	Injected faults are detected at the intended operation site.

Fuzzing	Hidden disagreement surfaces	Minimized disagreements become regression tests.
Baseline comparison	Unclear value over existing tools	Precision, recall, latency, and time-to-detect are compared against external alternatives.
Domain ablation	Unjustified abstract domains	Removing a domain changes measurable recall, precision, diagnostics, or runtime.
CEGAR depth ablation	Refinement over- or under-use	More refinement improves specific cases until convergence without unbounded runtime.
Operator coverage gate	Silent regression in transfer surface	Coverage and confidence tables remain in sync with code.
Deployment budget gate	Release-host regressions in export/compile checks	Pre/post export and compile gates stay within latency and memory budgets.
Deployment gallery	Architecture-specific export blind spots	Representative ResNet, ViT, Llama-style, diffusion, recommender, and speech blocks pass the same pre/post export gates.
Dashboard regeneration	Stale paper claims	Committed result artifacts match freshly regenerated outputs.
Statistical checks	Underpowered headline claims	Effect-size and sample-size artifacts document uncertainty.
Artifact packaging	Reproducibility friction	Docker, conda, source, and pipx paths can run the advertised checks.

E Adoption Scenarios

E.1 Local model authoring

A researcher writing a new module can run `tensorguard verify model.py` before starting training. The verifier infers shapes from common constructors, input hints, docstrings, and structural layer usage. If it finds a bad `Linear` feature count or attention mask, it reports a source-mapped diagnostic. If it reaches unsupported dynamic behavior, strict mode reports UNKNOWN with an explanation. This keeps the feedback loop close to the edit, not to the first expensive batch.

E.2 Continuous integration

In CI, teams can run high-confidence or sound mode on changed model files. The GitHub Action can annotate pull requests and SARIF can surface findings in code scanning. Baseline/suppression support lets a legacy repository adopt the tool incrementally: existing issues can be acknowledged while new regressions fail the build. Domain flags make ablation and staged rollout practical; for example, a team may initially enforce shape and dtype while logging phase diagnostics.

E.3 Notebook exploration

Jupyter/IPython integration checks a model cell when it is defined. This mode is intentionally lightweight. A notebook user benefits from early shape and device feedback without building a package or writing a config file. Because notebooks often contain dynamic or partially executed state, honest abstention is important: the tool should explain what it could not prove rather than pretend that notebook context is a static module.

E.4 Framework integration

Framework authors can call the public API before launching training, compiling a model, exporting an artifact, or serving a checkpoint. Hooks for Lightning, HuggingFace-style trainers, accelerate-like launchers, MLflow, W&B, and CI systems allow TensorGuard to act as a precondition gate. The value is not merely a shape checker; it is a common safety layer that can speak JSON, SARIF, JUnit, or native annotations.

E.5 Deployment gate

Deployment introduces contracts that ordinary training does not: ONNX opset availability, `torch.export` dynamic-shape `Dim` ranges and relations, ONNX-inexpressible derived dynamic dimensions, AOTInductor input layouts, dtype/device policies, `torch.compile` guard coverage, preserved dynamic guards and package-lowering denylists, quantization observer calibration, activation qscheme placement, eager/FX quant-dequant boundaries, mixed-precision dtype boundaries, AMP backend allowlists, fp16 GradScaler requirements, TorchServe or FastAPI request schemas, GGUF or llama.cpp checkpoint shapes, CUDA graph capture assumptions, and distributed shard and pipeline activation boundaries. TensorGuard's deployment modules check these contracts before the downstream system emits a harder-to-debug compiler, exporter, or serving error. For `torch.compile`, `src/compile_guard_analysis.py` makes the boundary auditable: it extracts TensorGuard's symbolic input and direct layer constraints, reads Dynamo guard code from `torch._dynamo.explain` when available, accepts recorded guard objects for deterministic CI, and reports the static facts that no runtime guard appears to protect. `evaluation/deployment_budgets.py` turns those checks into a release budget: FX runs before export and compile, `torch.export` runs after export graph capture, and Dynamo runs after compile capture on supported interpreters. The manifest records per-backend latency/memory Pareto curves, and the live gate fails if any supported row is unsafe or over budget. `evaluation/deployment_gallery.py` complements that budget gate with a model-family gallery: ResNet, ViT, Llama-style, diffusion U-Net, recommender, and speech modules are instantiated as real PyTorch code, executed to confirm their deployment input/output contract, then verified before and after export. The committed manifest records the model families, input specs, operator surfaces, and gate matrix; the live gate proves the gallery still passes on the current torch stack. For model serving, `src/serving_schema.py` splits a pure `verify_serving_schema` checker from guarded wrappers for FastAPI-style callables and TorchServe-style handlers. The pure gate validates materialized request, preprocessing, model-output, and response payloads against named specs with concrete, wildcard, or symbolic dimensions; the guarded path stops on request or preprocessing-output violations before model invocation. The regression tests run real `nn.Module`s and prove that an NHWC image batch is rejected against an NCHW `Conv2d` schema while the model's `forward` counter remains zero. For CUDA graph capture, `src/cuda_graph_capture.py` separates the capture eligibility proof from the capture body. It records static input signatures, checks replay shape/dtype/device/stride and optional storage-address reuse, scans the reachable `forward` helper closure for tensor factories, data-dependent shape producers such as `nonzero` and mask indexing, and host-synchronizing operations such as `item()`, then cross-checks the `torch.fx` graph against the same denylist. The guarded helper refuses to invoke

capture on violations, and the tests prove this on real `nn.Module` forwards while remaining runnable on CPU-only CI. For GGUF/llama.cpp, the gate reads the checkpoint tensors and metadata directly: it validates architecture-prefixed metadata, q/k/v or packed-qkv projections, rotary-pair dimensions, vocabulary and output projections, and per-quantization block-size divisibility without needing to run the external converter.

For training resume, TensorGuard now treats optimizer state as a first-class checkpoint contract. The gate maps live optimizer buffers by `Parameter` identity, validates AdamW and fused-AdamW moments against target parameter shapes, checks PyTorch Adafactor's factored `row_var`/`col_var` buffers, accepts legitimate lazy pre-step state, verifies declared ZeRO-style shard coverage, and rejects shape-incompatible checkpoint loads before `load_state_dict` mutates the optimizer. Dtype handling follows PyTorch's resume semantics: castable checkpoint dtypes are not rejected before load, but nonsensical live state dtypes remain actionable diagnostics.

For LoRA and PEFT fine-tuning, TensorGuard separates three concerns that PyTorch and PEFT normally discover only during load or inference. Adapter-only checkpoints are parsed using the HuggingFace key layout, live PEFT modules are inspected via their `lora_A`/`lora_B` dictionaries, and every discovered adapter is checked against the target base weight for rank agreement, input/output shape, declared `target_modules`, expected merged state, and QLoRA-style quantized-base placement with floating low-rank matrices. If only an adapter state is available, base-relative checks are explicitly reported as skipped rather than converted into a false proof.

E.6 Research artifact review

A reviewer can start from the soundness contract, run the new tests for any claimed operator family, inspect the evidence index, regenerate dashboards, and audit Lean files. This is a different posture from a one-off prototype: the paper claims are connected to code paths, tests, and artifacts, and unsupported regions are part of the documented result.

F Design Decisions Worth Preserving

Abstention is a feature. A static tensor verifier that guesses past unknown code can appear impressive on demos while being unusable in CI. TensorGuard instead treats UNKNOWN as a first-class outcome. This enables strict adoption: a user can decide whether an unknown is acceptable for their workflow.

Concrete bugs should be concrete. When a shape is known, the diagnostic should name the operation and the mismatched dimension. "Linear expects last dim=4, got 2" is more useful than "solver found SAT." Counterexamples are still valuable, but they should support the explanation rather than replace it.

Operator rules must match the live library. PyTorch changes, and some operator behavior is subtle. Transfer functions should be cross-checked against the installed PyTorch version wherever possible. Padding and chunk/split are model examples: intuitive symmetric padding or equal partitioning would be wrong, so the rules follow observed torch behavior.

Integration surfaces are part of correctness. A verifier that catches bugs only through an internal Python function is not enough. CI, SARIF, pre-commit, notebooks, watch mode, export gates, and framework hooks determine whether the analysis affects real development.

Evidence should regenerate. The repository's JSON artifacts, dashboards, coverage tables, and paper evidence files exist so claims can be checked. A best-paper-grade artifact must make it easy to see when a number or guarantee is stale.

Formalization should target the trust core. Proving all of PyTorch is not the goal. Proving the algebra that the implementation relies on, auditing axioms, and pairing proofs with conformance tests creates a tractable and useful trust story.

G Diagnostic Taxonomy

The user-facing product of the verifier is a diagnostic, not a theorem object. The following taxonomy keeps diagnostics comparable across domains and output formats.

Diagnostic class	Typical trigger	Useful response
Dimension mismatch	A contract expects two static dimensions to be equal and they are not.	Report both dimensions, the axis, and the operation that imposed the equality.
Rank mismatch	An operator requires a tensor rank or rank range that the input does not satisfy.	Report expected rank, observed rank, and whether a reshape, unsqueeze, or batch dimension is likely involved.
Product mismatch	A reshape, view, flatten, or unflatten changes the number of elements.	Report the source product, target product, and the unresolved symbolic factors if any.
Partition mismatch	Split, chunk, unbind, or unpacking returns a different number of values than the program expects.	For concrete cases, report the exact returned count; for symbolic cases, abstain with a dependency explanation.
Broadcast failure	Elementwise or mask shapes cannot be right-aligned under singleton expansion.	Report the first incompatible aligned axis and both input shapes.

Device conflict	Inputs to an operation reside on incompatible devices or an implicit host/device transfer would violate the selected policy.	Report the producer operation for each device and any missing explicit move.
Dtype conflict	Mixed dtypes are unsupported, lossy, or inconsistent with an operator's contract.	Report promoted or required dtype and the tensor whose dtype blocks the operation.
Gradient break	A training path detaches, casts, mutates, or guards tensors in a way that prevents intended gradient flow.	Report the first operation that breaks differentiability and the downstream loss or parameter it affects.
Optimizer-state mismatch	A resumed optimizer buffer has the wrong shape, dtype, factored Adafactor layout, or shard coverage for the target parameter.	Report the parameter, state key, expected buffer contract, and whether the problem blocks checkpoint loading or is only a lazy/master-state warning.
Export/serving precondition	ONNX, AOT, quantization, serving, CUDA graph, or accelerator-specific constraints are violated.	Report the backend-specific contract and the nearest portable replacement if known.
Unsupported dynamic region	The program constructs shapes or control flow from values that the selected analysis mode cannot resolve.	Return UNKNOWN with the unsupported expression and a suggestion for an input hint or annotation.
Baseline-suppressed issue	A finding exists but is acknowledged by an adoption baseline.	Keep the evidence in machine-readable output while not failing the incremental gate.
Evidence inconsistency	A dashboard, paper artifact, corpus label, or coverage table is stale relative to code or tests.	Fail regeneration checks and point to the artifact that must be rebuilt.

H Reviewer Checklist

A reviewer can inspect the trust boundary by reading `SOUNDNESS_CONTRACT.md`, `VERIFIABLE_FRAGMENT.md`, and `operator_confidence_table.json`. They can run the latest padding tests with:

```
python -m pytest -q tests/test_pad_semantics.py.
```

They can inspect the empirical story through `make reproduce`, the dashboard tests, real benchmark loaders, baseline comparisons, and the reproducibility directory. They can inspect formal claims through the Lean files and axiom-audit tests. Most importantly, they can observe that unsupported or symbolic-hard cases are not hidden: TensorGuard either proves, refutes, or says UNKNOWN with a reason.